COURSE NAME

SOFTWARE ENGINEERING

CSC 3114

(UNDERGRADUATE)

CHAPTER 1

SOFTWARE & SOFTWARE ENGINEERING

ARJUN KUMAR BOSE ARNOB
LECTURER, CS, AIUB
arnob@aiub.edu

WHAT IS SOFTWARE ENGINEERING?

- Technologies that make it easier, faster, and less expensive to build high-quality computer programs
- A discipline aiming at producing fault-free software, delivered on time and within budget, that satisfies the users' needs.

- An engineering: A set of activities in software production
- The philosophy of established engineering disciplines to solve what are termed software crisis

SCOPE OF SOFTWARE ENGINEERING

- The aim of Software Engineering is to solve Software Crisis:
  - Late
  - Over budget
  - Low quality with lots of faults

- Software crisis is still present over 35 years later!

SOFTWARE CHARACTERISTICS

- A logical (intangible) rather than a physical system element
- Being developed or engineered but not being manufactured
- Software cost concentrating in engineering, not in materials
- Software does not "wearing out" but "deteriorating"(not destroyed after lifetime like hardware, but backdated by aging that needs to update)
- Software is a 'differentiator' (different sub-systems, e.g. cashier's workstation in a supermarket)
- Without "spare parts" in software maintenance (no extra useless features in software)
- Most software continues to be custom-built (based on the requirements)

SOFTWARE DEVELOPMENT LIFE CYCLE (SDLC)

- A structured set of activities required to develop a software system
- The way we produce software, including:

1. Planning
2. Requirements Analysis
3. Design
4. Development
5. Testing
6. Final release / Deployment
7. Maintenance

SOFTWARE DEVELOPMENT LIFE CYCLE (SDLC)

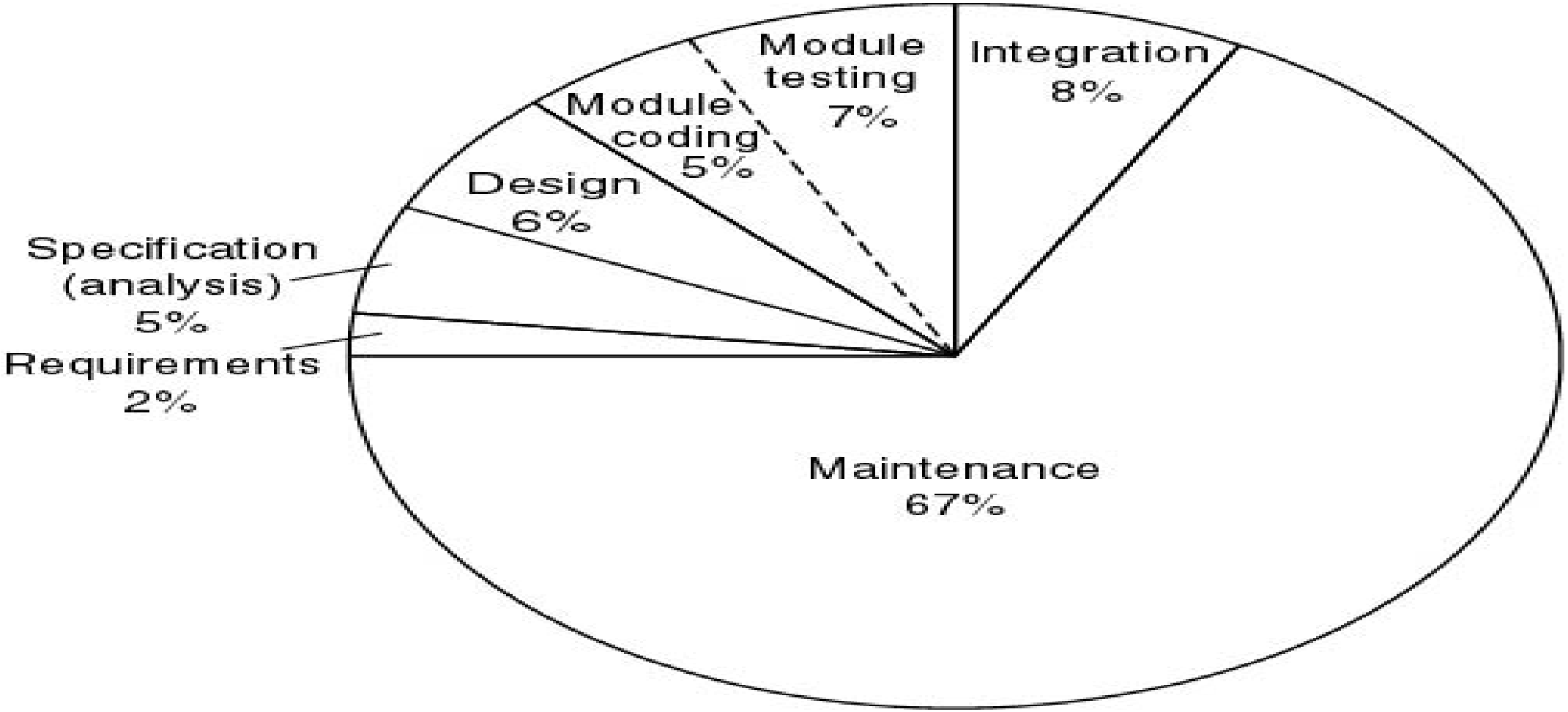


GOOD & BAD SOFTWARE

- Good software is maintained—bad software is discarded

- Different types of maintenance
  - Corrective maintenance [about 20%]
    - Modification to fix a problem
  - Enhancement [about 80%]
    - Perfective maintenance (modification to improve usability,…) [about 60%]
    - Adaptive maintenance (modification to keep up-to-date) [about 20%]
    - Preventive maintenance (modification to avoid any future error) [about 20%]

COST OF DETECTION & CORRECTION OF A FAULT



SOFTWARE APPLICATION

- System software (control computer hardware such as OS)
- Business software (commercial application for business users [SAP, ERP])
  - Systems, Applications, and Products
  - Enterprise Resource Planning
- Engineering and scientific software (e.g. statistical analysis-SPSS, MATLAB)
- Embedded software (e.g. biometric device)
- Personal computer software (e.g. Microsoft Office)
- Web-based software (use over the internet with a browser, e.g. Gmail)
- Artificial intelligence software (e.g. HCI, Google AI)

SOFTWARE MYTHS (MANAGEMENT)

- Myth 1: We already have a book full of standards and procedures for building software. Wouldn't that provide my people with everything they need to know?
- Myth 2: My people have state-of-the-art software development tools; we buy them the newest computers.
- Myth 3: If we get behind schedule, we can add more programmers and catch up.
- Myth 4: If I outsource the software project to a third party, I can relax and let that firm build it.

SOFTWARE MYTHS (CLIENT)

- Myth 1: A general statement of objectives is sufficient to begin writing programs – we can fill in the details later.

- Myth 2: Project requirements continually change, but change can be easily accommodated because software is flexible.

SOFTWARE MYTHS (PROFESSIONAL)

- Myth 1: Our job is done once we write the program and get it to work.

  Fact: the sooner you begin writing code, the longer it will take you to get done.

- Myth 2: I cannot assess the program's quality until it is "running. "

- Myth 3: The working program is the only deliverable work product for a successful project.

- Myth 4: Software engineering will make us create very large and unnecessary documentation, invariably slowing us down.

WHY SYSTEM FAILS?

- The system fails to meet the business requirements for which it was developed. The system is either abandoned or expensive adaptive maintenance is undertaken.
- There are performance shortcomings in the system, which make it inadequate for the users' needs. Again, it is either abandoned or amended incurring extra costs.
- Errors appear in the developed system causing unexpected problems. Patches have to be applied at extra cost.
- Users reject the implemented system, lack of involvement in its development or lack of commitment to it.
- Systems are initially accepted but over time become un-maintainable and so pass into disuse.

REFERENCES

- R.S. Pressman & Associates, Inc. (2010). *Software Engineering: A Practitioner's Approach*.
- Kelly, J. C., Sherif, J. S., & Hops, J. (1992). An analysis of defect densities found during software inspections. *Journal of Systems and Software*, *17*(2), 111-117.
- Bhandari, I., Halliday, M. J., Chaar, J., Chillarege, R., Jones, K., Atkinson, J. S., & Yonezawa, M. (1994). In-process improvement through defect data interpretation. *IBM Systems Journal*, *33*(1), 182-214.